Devstack — Multi-PHP Local Development Environment

Complete step-by-step documentation

Target: Apple Silicon (arm64) Mac · **Stack:** Docker **Location:** `~/devstack`

This document records, in order, how to set up a Docker-based environment running **7 PHP versions at once** (7.2 – 8.4) with a shared MySQL 8.0, phpMyAdmin, Node (via fnm), clean `.test` hostnames, and trusted HTTPS — plus how to run a real Laravel project on it.

Table of contents

---

1. Prerequisites

- **Homebrew**
- **Docker Desktop** (must be running — start with `open -a Docker`)
- Native `arm64` builds exist for all PHP images (7.2–8.4), so no emulation is needed.

---

2. Design decisions

Question	Choice	Why
How to manage 7 PHP versions	**Docker, one shared stack**	EOL versions (7.2/7.3) "just work"; matches prod; run all at once
Do projects get dockerized individually	**No**	Projects are plain folders in `www/` ; nginx routes to the right PHP
Location	`~/devstack`	self-contained
Node	**fnm**	fast, switch Node per project
Database	**MySQL 8.0**	modern, compatible with legacy apps

Key idea: **one nginx + one MySQL + 7 PHP-FPM containers.** A project is just a folder; the PHP version is chosen by the URL you use.

---

3. Directory structure

```
~/devstack/
├── docker-compose.yml          # all services
├── .env                        # MySQL creds + timezone (gitignored)
├── php/
│   ├── Dockerfile              # one image, reused per version via build arg
│   └── conf.d/zz-devstack.ini  # shared PHP settings (upload size, xdebug off…)
├── nginx/
│   ├── conf.d/                 # one vhost per version + per-project vhosts
│   │   └── laravel-example.conf.example   # copy this for a Laravel app
│   └── certs/                  # mkcert TLS cert + key (gitignored)
├── www/                        # ← your projects live here
│   └── info/                   # sample project (phpinfo + DB check)
├── data/mysql/                 # persistent MySQL data (gitignored)
├── bin/stack                   # helper CLI (symlinked to /opt/homebrew/bin/stack)
├── README.md
└── DOCUMENTATION.md            # this file
```

---

4. The PHP images (7.2–8.4)

A **single `php/Dockerfile`** is reused for every version via a build arg. It uses `mlocati/php-extension-installer`, which handles extension/version/arch differences uniformly across 7.2 → 8.4:

```
ARG PHP_VERSION=8.2
FROM php:${PHP_VERSION}-fpm
COPY --from=mlocati/php-extension-installer /usr/bin/install-php-extensions /usr/local/bin/
RUN install-php-extensions \
      pdo_mysql mysqli gd zip intl bcmath mbstring opcache exif pcntl soap xdebug @composer
WORKDIR /var/www/html
```

Every version therefore includes: **Composer**, common extensions, and **Xdebug** (installed but `mode=off` by default — see `php/conf.d/zz-devstack.ini`).

Versions built: **7.2, 7.3, 7.4, 8.0, 8.1, 8.2, 8.4** (no 8.3).

> **Gotcha:** building all 7 in parallel exhausts memory and randomly fails (`EOF` / killed). Fix: build failing versions **one at a time** (`docker compose build php81`).

---

5. docker-compose stack

Services (`docker-compose.yml`):

- `php72 … php84` — 7 PHP-FPM containers, each built from the shared Dockerfile
- `nginx` — routes requests to the right PHP version
- `mysql` — MySQL 8.0, data persisted to `./data/mysql`
- `phpmyadmin` — web DB admin

Credentials live in `.env` (copy from `.env.example`):

```
MYSQL_ROOT_PASSWORD=root
MYSQL_DATABASE=app
MYSQL_USER=app
MYSQL_PASSWORD=app
TZ=UTC
```

---

6. nginx routing

Each version has a vhost in `nginx/conf.d/phpNN.conf` with three server blocks:

1. **port 80** → 301 redirect to HTTPS (for `*.NN.test`)
2. **port 443** → serves the project over TLS
3. **`localhost:80NN`** → plain-HTTP no-DNS fallback

Access map:

PHP	HTTPS hostname	Plain-HTTP fallback
7.2	`https://<proj>.72.test`	`http://localhost:8072/<proj>/`
7.3	`https://<proj>.73.test`	`http://localhost:8073/<proj>/`
7.4	`https://<proj>.74.test`	`http://localhost:8074/<proj>/`
8.0	`https://<proj>.80.test`	`http://localhost:8080/<proj>/`
8.1	`https://<proj>.81.test`	`http://localhost:8081/<proj>/`
8.2	`https://<proj>.82.test`	`http://localhost:8082/<proj>/`
8.4	`https://<proj>.84.test`	`http://localhost:8084/<proj>/`

Other services: **phpMyAdmin** `http://localhost:8888` · **MySQL** `127.0.0.1:3306`

---

7. Building & starting

```
# 1. Start Docker Desktop (must be running)
open -a Docker

# 2. Build the 7 PHP images (build one at a time if parallel fails)
cd ~/devstack && docker compose build

# 3. Start everything
docker compose up -d
```

Every version reports the correct `PHP x.y.z` and connects to `MySQL 8.0` (tested via the `info` sample project).

8. The `stack` helper command

`bin/stack`, symlinked to `/opt/homebrew/bin/stack` (on PATH):

```
stack new <ver> <name> [laravel]   # scaffold a project folder + nginx vhost
stack up                       # start everything
stack down                     # stop
stack restart                  # restart containers
stack rebuild                  # rebuild PHP images + restart
stack ps                       # status
stack logs [service]           # tail logs

stack php <ver> <project> ...      # run php in a project    e.g. stack php 8.2 myapp -v
stack composer <ver> <project> ... # composer               e.g. stack composer 7.4 shop install
stack artisan <ver> <project> ...  # laravel artisan        e.g. stack artisan 8.2 myapp migrate
stack sh <ver>                     # shell into a PHP container
stack mysql                        # mysql client as root
```

(`<ver>` accepts `8.2` or `82`.)

9. Node.js via fnm

```
brew install fnm
# add to ~/.zshrc:
eval "$(fnm env --use-on-cd --version-file-strategy=recursive)"
fnm install --lts        # e.g. Node v24 LTS
```

Node runs **on the host** (not in the PHP containers). `--use-on-cd` auto-switches Node version when a project has a `.node-version` / `.nvmrc` file.

10. Clean `.test` hostnames (dnsmasq)

Makes every `*.test` domain resolve to `127.0.0.1` with no `/etc/hosts` edits.

```
brew install dnsmasq
# config:
#   /opt/homebrew/etc/dnsmasq.d/devstack-test.conf  ->  address=/.test/127.0.0.1
#   (conf-dir line added to /opt/homebrew/etc/dnsmasq.conf)
```

Manual step (needs sudo — run in a real Terminal):

```
sudo brew services start dnsmasq
sudo mkdir -p /etc/resolver
echo "nameserver 127.0.0.1" | sudo tee /etc/resolver/test
sudo dscacheutil -flushcache && sudo killall -HUP mDNSResponder
```

Verify: `info.82.test` etc. resolve to `127.0.0.1`. Survives reboots.

11. Trusted HTTPS (mkcert)

```
brew install mkcert nss
# one cert covering every version's wildcard + localhost:
cd ~/devstack/nginx/certs
mkcert -cert-file devstack.pem -key-file devstack-key.pem \
  "*.72.test" "*.73.test" "*.74.test" "*.80.test" "*.81.test" "*.82.test" "*.84.test" \
  localhost 127.0.0.1
```

- Cert + key are mounted into nginx at `/etc/nginx/certs/` (port 443 exposed).
- Wildcards are **one level deep** (`a.82.test` works, `a.b.82.test` does not).

Manual step (needs sudo — run in a real Terminal), to trust the CA in browsers:

```
mkcert -install
```

Then **fully quit and reopen the browser**.

> If the browser says "Not secure", it's almost always because `mkcert -install` hasn't been run — verify with: `security find-certificate -c "mkcert" /Library/Keychains/System.keychain`

---

12. HTTP → HTTPS redirect

Every `.test` vhost returns **301** → **https** on port 80; the `localhost:PORT` fallbacks stay plain HTTP (no TLS on those ports by design).

Verified:

- `http://<proj>.82.test` → `301` → `https://<proj>.82.test` → `200`
- `http://localhost:8082/info/` → stays `200` (plain HTTP)

---

13. Example: running a Laravel project

Say you have a Laravel app to run on PHP 8.2. Call it `myapp` .

```
# 1. Put the code in the www folder
cd ~/devstack/www
git clone <your-repo-url> myapp        # or copy the project in

# 2. Install dependencies at the right PHP version
stack composer 8.2 myapp install
#    Legacy project with old, advisory-flagged packages? Temporarily allow them:
#      composer config policy.advisories.block false && composer install
#    (revert composer.json afterwards to keep your tree clean)

# 3. Add an nginx vhost (copy the example template; .local.conf keeps it out of git)
cp nginx/conf.d/laravel-example.conf.example nginx/conf.d/myapp.local.conf
#    edit: server_name myapp.82.test; root .../myapp/public; fastcgi_pass php82:9000;
stack restart

# 4. Create the database
stack mysql
#    then: CREATE DATABASE myapp; GRANT ALL ON myapp.* TO 'app'@'%'; FLUSH PRIVILEGES;

# 5. Configure .env (point Laravel at the shared MySQL)
#      APP_URL=https://myapp.82.test
#      DB_HOST=mysql
#      DB_DATABASE=myapp
#      DB_USERNAME=app
#      DB_PASSWORD=app
stack php 8.2 myapp artisan key:generate

# 6. Bring the schema up (fresh migrate OR import a dump)
stack artisan 8.2 myapp migrate
#    — or, if the project is normally seeded from a DB dump:
#      docker exec -i devstack-mysql-1 mysql -uroot -proot myapp < dump.sql

# 7. Laravel storage dirs are gitignored — create + make writable after clone
mkdir -p www/myapp/storage/framework/{sessions,views,cache/data} \
         www/myapp/storage/logs www/myapp/bootstrap/cache
chmod -R 777 www/myapp/storage www/myapp/bootstrap/cache

# 8. Clear caches
stack php 8.2 myapp artisan config:clear
```

Open `https://myapp.82.test`.

Common real-world gotchas

- **Migrations that don't run from scratch** (e.g. foreign keys referencing columns a later migration adds): the project is likely meant to be set up from a **DB dump** rather than fresh migrations.
- **Service providers that query the DB at boot**: nothing (not even `artisan`) works until the database is populated. Import data first.
- **Composer refuses old packages** flagged by security advisories: `composer config policy.advisories.block false`.
- `storage/framework/*` **missing after clone**: it's gitignored — recreate it.
- `git` **isn't inside the PHP containers**: run git on the host.

---

14. Daily usage cheat sheet

```
stack up                # morning: start the stack
stack ps                # what's running
stack down              # end of day: stop (data persists)

# a project
open https://myapp.82.test
stack artisan 8.2 myapp migrate:status
stack composer 8.2 myapp install

# database
open http://localhost:8888          # phpMyAdmin (root/root or app/app)
stack mysql                          # CLI
```

15. Adding a new project

Fastest — use the scaffolder:

```
stack new 8.2 <name>            # plain PHP project
stack new 7.4 <name> laravel    # Laravel (web root = /public)
```

It creates `www/<name>/`, writes `nginx/conf.d/<name>.local.conf`, and reloads nginx. Then put your code in the folder and open the printed URL.

Manually — plain PHP project:

1. Put code in `~/devstack/www/<name>/`
2. Open `https://<name>.82.test` (or any version), or `http://localhost:8082/<name>/`

Manually — Laravel/Symfony project (web root is `/public`):

1. Put code in `~/devstack/www/<name>/`
2. `cp nginx/conf.d/laravel-example.conf.example nginx/conf.d/<name>.local.conf`, edit `server_name`, `root .../public`, and `php82` → desired version
3. `stack restart`
4. `stack composer <ver> <name> install`, set up `.env` (`DB_HOST=mysql`), create/import the DB
5. Open `https://<name>.82.test`

HTTPS works automatically for any new `*.NN.test` host — no cert regeneration.

16. Troubleshooting & gotchas

Symptom	Cause / Fix
`Cannot connect to the Docker daemon`	Docker Desktop not running → `open -a Docker`
PHP image build fails randomly (`EOF`)	Parallel build OOM → build one at a time
`bind: address already in use`	Another project uses the same host port → change its host port or `stack down`
Browser "Not secure" on `*.test`	`mkcert -install` not run, or browser not restarted
"Not secure" on `localhost:PORT`	Expected — those are plain HTTP by design; use the `.test` hostname
Laravel 500 `Failed to open stream: .../sessions`	`storage/framework/*` missing → `mkdir -p` + `chmod -R 777 storage bootstrap/cache`
Laravel DB errors at boot	DB empty/not imported, or `.env` `DB_HOST` wrong (should be `mysql`)
Composer refuses to install old packages	`composer config policy.advisories.block false`
`git` not found inside container	Run git on the **host**

---

17. Manual (sudo) steps

These require your admin password, so run them in a **real Terminal** (not a non-interactive shell):

1. **dnsmasq activation** — see §10.
2. **Trust the HTTPS CA** (fixes "Not secure"):

   ```
   mkcert -install
   ```

   then fully quit & reopen your browser.

Credential hygiene

When cloning private repos, authenticate once in your own terminal so the credential is stored in the macOS Keychain — don't hard-code tokens into `.git/config` or commit them. `.env` , `nginx/certs/` , and `data/` are gitignored for this reason.

End of documentation.